

EE 467 Homework 1: Twitter Spam Detection

Welcome to the first homework of EE 467! Here we will apply the ML pipeline introduced in the second lab to **Twitter spam message detection problems**. In the past few years, social medias like Twitter has become the target of spam posting. Automatic spam posts can not only affect user experience, but they can also spread misinformation and subtly affect public opinions.

In this homework, we will work on a mini Twitter message dataset extracted from the CRESCEI-2017 dataset. This dataset contains 50000 Twitter messages, half of which are sent by real Twitter users and half of which generated by spam bots. Your task is to train classifiers that can tell these two kinds of messages apart. This time, you'll go deeper and try multiple feature extractors and classifiers. Like the labs, you will need to fill out blanks in code cells. You are also going to answer a few questions and **optionally** work on improvements to existing pipeline completely on your own.

Before You Start

- You can discuss on Slack if you have questions and want to seek help; however, please try your best to **limit the scope of your question and avoid asking directly for answers**. You should also **avoid copy-pasting answers and code** from others.
- You **may use AI assistants** (e.g., ChatGPT, Gemini, etc.) to support your work. If you do, you must **clearly acknowledge** this by stating:
 - The name of the AI assistant**, and
 - How it was used** (e.g., brainstorming, clarifying concepts, debugging, editing, generating examples)

You **must not copy and paste** AI-generated content directly into your submission without **fully understanding** what it does and being able to **explain it in your own words**.

- You can **optionally work on this homework with one other student of this course as a team**, but **each of you needs to submit the homework individually**. If you choose to form a group, please include the name of your teammate here: *(Name of your teammate)*.

Feature Extraction

We will begin our ML pipeline with feature extraction in this homework, as the dataset is good enough and does not need any special pre-processing. As usual, we will load the dataset into memory, extract samples features into vectors and split the dataset into training and test sets.

```
In [13]: import pandas as pd

# 1) Load all data from `twitter-mini.csv` into variable `data`
data = pd.read_csv("../twitter-mini/twitter-mini.csv")
# 2) Preview first few samples
print(data.head())
```

	text	label
0	@baydiangirl it definitely moved some church f...	0
1	@suebob Looks like #TXlege made up new rules f...	0
2	RT @saleemchat: #WJChat #A18 5 yrs: People ski...	0
3	@SajidaBalouch Huhhhh	0
4	@Dukester_94 im the lad of all bibles	0

In lab 2 we have learned how to use the `CountVectorizer` feature extractor, which counts the occurrence of each token (letter here) in a sample (Twitter message text here). This time, we will also try `TfidfVectorizer` and see which feature extractor performs better. Here we will introduce the algorithm it implemented, **TF-IDF**:

TF-IDF is the abbreviation for **term frequency-inverse document frequency**, and is defined as the product of the two frequency. The most classical version of TF-IDF is:

$$tf-idf(t) = tf(t, d) \cdot idf(t, D) = \frac{f_{t,d}}{|d|} \log \frac{|D|}{|D_t|}$$

Where t is the term, D is the collection of all documents, d is one of the document that contains t and D_t is the subset of documents that contains term t . The advantage of TF-IDF is that frequent and useless words, such as "the", "a" and "an" will receive lower weight, since they appear in almost all documents. On the other hand, unique yet frequent (in any $d \in D$) words will get higher weight. Research has shown that TF-IDF is easy to implement and performs relatively well.

Before we run these feature extractors on our dataset, we need to convert each message into a sequence of terms. Since Twitter messages often contain emojis, abbreviations, hashtags and other composition of characters, we can't use the word-level analyzer as we have done in lab 2. Instead, we will generate **character N-grams** from these messages.

```
In [14]: from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer
# 1) Extract features of all messages into `feat1` with `CountVectorizer` using `CHARACTER` trigrams
# 2) Repeat with `TfidfVectorizer` and save generated features as `feat2`
print("Extracting features with `CountVectorizer`...")
vectorizer = CountVectorizer(ngram_range=(3,3), analyzer='char')
feat1 = vectorizer.fit_transform(data['text'])

print("Extracting features with `TfidfVectorizer`...")
tvectorizer = TfidfVectorizer(analyzer='char', ngram_range=(3,3))
feat2 = tvectorizer.fit_transform(data['text'])
print("Completed.")

print(feat1.shape)
print(feat2.shape)
```

Extracting features with `CountVectorizer` ...
Extracting features with `TfidfVectorizer` ...
Completed.
(50000, 79578)
(50000, 79578)

We will also try combining two kinds of feature generated above into a single feature. We will use `scipy.sparse.hstack` to concatenate corresponding features for each sample. Remember, concatenated features matrix should have same amount of samples as `feat1` and `feat2`.

```
In [15]: from scipy.sparse import hstack
# Concatenate token count features and TF-IDF features on the feature dimension,
# name the combined feature matrix `feat_comb`.
feat_comb = hstack([feat1, feat2])
print(feat_comb.shape)

(50000, 159156)
```

Finally, let's take a look at the labels. In our dataset, 0 represents messages from real Twitter users and 1 represents spam messages.

```
In [11]: y = data["label"].values
print(y[:5])

[0 0 0 0 0]
```

Training

As usual, we will start by doing a 80% / 20% training-test split. Like lab 2, we suggest you to **use the same random seed for all splits** for comparable and reproducible results.

```
In [16]: from sklearn.model_selection import train_test_split
# 1) Split token count features into 80/20 training and test sets
# 2) Split TF-IDF features into 80/20 training and test sets
# 3) Split combined features into 80/20 training and test sets

split_seed = 1223
# Token count features
print("Splitting token count features...")
tokenx_train, tokenx_test, tokeny_train, tokeny_test = train_test_split(feat1, y, test_size = 0.2, random_state=split_seed)

# TF-IDF features
print("Splitting TF-IDF features...")
tfx_train, tfx_test, tfy_train, tfy_test = train_test_split(feat2, y, test_size = 0.2, random_state=split_seed)

# Combined features
print("Splitting combined features...")
combx_train, combx_test, comby_train, comby_test = train_test_split(feat_comb, y, test_size = 0.2, random_state=split_seed)
print("Completed.")
```

Splitting token count features...
Splitting TF-IDF features...
Splitting combined features...
Completed.

We are now ready to train **binary classifiers** for spam message detection. Apart from the `LogisticRegression` classifier you have already seen and used during Lab 1, we will try a few more classifiers:

- MultinomialNB** (Naive Bayes):

This classifier is based on the **Naive Bayes** assumption that, *given the class label*, individual features are conditionally independent.

For text classification, `MultinomialNB` is used with **token count** or **TF-IDF-like** features.

It models the probability of a message belonging to a class $y \in \{\text{spam}, \text{ham}\}$ as:

$$P(y \mid \mathbf{x}) \propto P(y) \prod_{j=1}^d P(x_j \mid y)$$

where $\mathbf{x} = (x_1, x_2, \dots, x_d)$ is the feature vector (e.g., word counts), and $P(x_j \mid y)$ is learned from training data. The predicted label is chosen using:

$$y_{\text{pred}} = \arg \max_y P(y \mid \mathbf{x})$$

In practice, the computation is done in the **log domain** to avoid numerical underflow:

$$y_{\text{pred}} = \arg \max_y \left(\log P(y) + \sum_{j=1}^d x_j \log P(x_j \mid y) \right)$$

- LinearSVC**:

This classifier is based on **linear SVM**, a popular algorithm for binary classification. The linear SVM algorithm treats the feature vector of each sample as a point in a linear space, and tries to find a hyperplane that separates points of the two classes as well as possible.

In practice, we optimize a **squared hinge loss with regularization**, since the features may not be perfectly linearly separable:

$$l_{\text{hinge}} = \max(0, y(1 - \mathbf{x}^T \mathbf{w}))^2 + \beta \|\mathbf{w}\|_2^2$$

Similarly, the classification result is determined by:

$$y_{\text{pred}} = \text{sign}(1 - \mathbf{x}^T \mathbf{w})$$

We have three kinds of features (**token count**, **TF-IDF**, and **combined**) and three kinds of classifiers. Here we will only try three combinations and compare their performance:

- `MultinomialNB` on token count features
- `LogisticRegression` on TF-IDF features
 - Try increasing the number of iterations if a warning of non-convergence pops up
- `LinearSVC` on combined features
 - For linear SVM, you can ignore the non-convergence warning as it does not affect performance

```
In [17]: from sklearn.naive_bayes import MultinomialNB
from sklearn.linear_model import LogisticRegression
from sklearn.svm import LinearSVC
# 1) Train a `MultinomialNB` classifier called `model1` on token count training set
# 2) Train a `LogisticRegression` classifier called `model2` on TF-IDF training set
# 3) Train a `LinearSVC` classifier called `model_comb` on combined features training set

print("Training Naive Bayes classifier on token count training set...")
model1 = MultinomialNB()
model1.fit(tokenx_train, tokeny_train)

print("Training logistic regression classifier on TF-IDF training set...")
model2 = LogisticRegression()
model2.fit(tfx_train, tfy_train)

print("Training SVM classifier on combined features training set...")
model_comb = LinearSVC()
model_comb.fit(combx_train, comby_train)

print("Completed.")
```

Training Naive Bayes classifier on token count training set...
Training logistic regression classifier on TF-IDF training set...
Training SVM classifier on combined features training set...
Completed.
c:\Users\kelly\anaconda\lib\site-packages\sklearn\svm_base.py:1249: ConvergenceWarning: Liblinear failed to converge, increase the number of iterations.
warnings.warn(

Evaluation

Finally, it's time to evaluate our models. You probably noticed that we have repeated similar code for different feature extraction and classification models. To avoid that, we will use `for` loops to make evaluation simpler:

```
In [19]: from sklearn.metrics import classification_report, confusion_matrix, accuracy_score
features = [{"Token count", feat1}, ("TF-IDF", feat2), ("Combined", feat_comb)]
models = [{"Naive Bayes", model1}, ("Logistic", model2), ("SVM", model_comb)]

# Complete missing code in the loop body
for i, ((feat_name, feat), (model_name, model)) in enumerate(zip(features, models)):
    # Begin of metrics for combination
    print("=====")
    print(f"#{i+1}. {model_name} on {feat_name}:\n")

    # 1) Split features into training and test set
    # (Note: you should use the same random number seed as the one in the training-test split above)
    x_train, x_test, y_train, y_test = train_test_split(feat, y, test_size = 0.2, random_state=split_seed)

    # 2) Predict labels for training data
    pred_train = model.predict(x_train)

    # 3) Print metrics for training set (precision, recall and F1 metrics)
    print("Training metrics:", classification_report(y_train, pred_train))
    # Confusion matrix
    print("Training confusion matrix:\n", confusion_matrix(y_train, pred_train), "\n")
    # Accuracy
    print("Train accuracy:", accuracy_score(y_train, pred_train), "\n")

    # 4) Predict labels for test data
    pred_test = model.predict(x_test)

    # 5) Print metrics for test set (precision, recall and F1 metrics)
    print("Test metrics:", classification_report(y_test, pred_test))
    # Confusion matrix
    print("Test Confusion Matrix:\n", confusion_matrix(y_test, pred_test), "\n")
    # Accuracy
    print("Test Accuracy:", accuracy_score(y_test, pred_test))

    # End of metrics for combination
    print("=====")

=====
1. Naive Bayes on Token count:

Training metrics:           precision    recall  f1-score   support

   0       0.92      0.99      0.95     19912
   1       0.99      0.91      0.95      2088

 accuracy          0.95      0.95      0.95     40000
macro avg          0.95      0.95      0.95     40000
weighted avg       0.95      0.95      0.95     40000

Training confusion matrix:
[[19652  260]
 [ 1718 18370]]

Train accuracy: 0.95055

Test metrics:           precision    recall  f1-score   support

   0       0.91      0.98      0.94      5088
   1       0.98      0.90      0.94      4912

 accuracy          0.94      0.94      0.94     10000
macro avg          0.94      0.94      0.94     10000
weighted avg       0.94      0.94      0.94     10000

Test Confusion Matrix:
[[4991  97]
 [ 492 4420]]

Test Accuracy: 0.9411
=====
2. Logistic on TF-IDF:

Training metrics:           precision    recall  f1-score   support

   0       0.94      0.99      0.97     19912
   1       0.99      0.94      0.97      2088

 accuracy          0.97      0.97      0.97     40000
macro avg          0.97      0.97      0.97     40000
weighted avg       0.97      0.97      0.97     40000

Training confusion matrix:
[[19778  134]
 [ 1179 18909]]

Train accuracy: 0.967175

Test metrics:           precision    recall  f1-score   support

   0       0.93      0.98      0.95      5088
   1       0.98      0.92      0.95      4912

 accuracy          0.95      0.95      0.95     10000
macro avg          0.95      0.95      0.95     10000
weighted avg       0.95      0.95      0.95     10000

Test Confusion Matrix:
[[5009  79]
 [ 399 4513]]

Test Accuracy: 0.9522
=====
3. SVM on Combined:

Training metrics:           precision    recall  f1-score   support

   0       1.00      1.00      1.00     19912
   1       1.00      1.00      1.00      2088

 accuracy          1.00      1.00      1.00     40000
macro avg          1.00      1.00      1.00     40000
weighted avg       1.00      1.00      1.00     40000

Training confusion matrix:
[[19912   0]
 [  22 20066]]

Train accuracy: 0.99945

Test metrics:           precision    recall  f1-score   support

   0       0.94      0.94      0.94      5088
   1       0.94      0.94      0.94      4912

 accuracy          0.94      0.94      0.94     10000
macro avg          0.94      0.94      0.94     10000
weighted avg       0.94      0.94      0.94     10000

Test Confusion Matrix:
[[4782  306]
 [ 282 4630]]

Test Accuracy: 0.9412
=====
```

1. Naive Bayes on Token count:				
Training metrics:	precision	recall	f1-score	support
0	0.92	0.99	0.95	19912
1	0.99	0.91	0.95	2088
accuracy	0.95	0.95	0.95	40000
macro avg	0.95	0.95	0.95	40000
weighted avg	0.95	0.95	0.95	40000
Training confusion matrix:				
[[19652 260]				
[1718 18370]]				
Train accuracy: 0.95055				
Test metrics:	precision	recall	f1-score	support
0	0.91	0.98	0.94	5088
1	0.98	0.90	0.94	4912
accuracy	0.94	0.94	0.94	10000
macro avg	0.94	0.94	0.94	10000
weighted avg	0.94	0.94	0.94	10000
Test Confusion Matrix:				
[[4991 97]				
[492 4420]]				
Test Accuracy: 0.9411				
=====				
2. Logistic on TF-IDF:				
Training metrics:	precision	recall	f1-score	support
0	0.94	0.99	0.97	19912
1	0.99	0.94	0.97	2088
accuracy	0.97	0.97	0.97	40000
macro avg	0.97	0.97	0.97	40000
weighted avg	0.97	0.97	0.97	40000
Training confusion matrix:				
[[19778 134]				
[1179 18909]]				
Train accuracy: 0.967175				
Test metrics:	precision	recall	f1-score	support
0	0.93	0.98	0.95	5088
1	0.98	0.92	0.95	4912
accuracy	0.95	0.95	0.95	10000
macro avg	0.95	0.95	0.95	10000
weighted avg	0.95	0.95	0.95	10000
Test Confusion Matrix:				
[[5009 79]				
[399 4513]]				
Test Accuracy: 0.9522				
=====				
3. SVM on Combined:				
Training metrics:	precision	recall	f1-score	support
0	1.00	1.00	1.00	19912
1	1.00	1.00	1.00	2088
accuracy	1.00	1.00	1.00	40000
macro avg	1.00	1.00	1.00	40000
weighted avg	1.00	1.00	1.00	40000
Training confusion matrix:				
[[19912 0]				
[22 20066]]				
Train accuracy: 0.99945				
Test metrics:	precision	recall	f1-score	support
0	0.94	0.94	0.94	5088
1	0.94	0.94	0.94	4912
accuracy	0.94	0.94	0.94	10000
macro avg	0.94	0.94	0.94	10000
weighted avg	0.94	0.94	0.94	10000
Test Confusion Matrix:				
[[4782 306]				
[282 4630]]				
Test Accuracy: 0.9412				
=====				

Questions

Please answer the following questions. You can just write down the answer below each question in this cell.

- Please compute **tf-idf** of word-level **bigrams** for the following three messages **by hand**. You should use the TF-IDF definition introduced in class.

- Message 1: "vegetables are good for health"
 - Message 2: "fruits are good source of vitamins"
 - Message 3: "proteins are good for health"
- Compute and sort intermediate and final results for **all occurring bigrams in the following order**:

["Vegetables are", "are good", "good for", "for health", "fruits are", "good source", "source of", "of vitamins", "proteins are"]

Then complete the following questions:

- Term Frequency (**tf**) of **bigrams** for **message 2**.
- Inverse Document Frequency (**idf**) of **bigrams** for **message 2**.
- tf-idf** of **bigrams** for **message 2**.
- Is the **tf-idf** value for the term "are good" equal to 0? If it's 0, can you explain why it makes sense to have 0 for that term?
- E. If we consider unigram (1-gram) based bag of words, how many words in these three messages would be assigned a 0 value **tf-idf**? And please explain why.

- What does `support` mean in the `classification_report`?

- If the training data is highly unbalanced, is `accuracy` a good metric for the classifiers' performance? Does a high accuracy necessarily means a high f1-score? Compare `accuracy` with `f1-score`. Based on your criterion, which type of classifier is better? Please give some justifications to support your answer.

- For most cases, the amount of bad data is much less than good data. How can you mitigate the data imbalance issue?

1

["vegetables are", "are good", "good for", "for health", "fruits are", "good source", "source of", "of vitamins", "proteins are"]

Message 1: [0.25, 0.25, 0.25, 0.25, 0, 0, 0, 0, 0] [ln(3), ln(1), ln(3/2), ln(3/2), ln(3), ln(3), ln(3), ln(3)] = [0.27, 0, 0.1, 0.1, 0, 0, 0, 0]

Message 2: [0, 0.2, 0, 0, 0.2, 0.2, 0.2, 0.2, 0] [ln(3), ln(1), ln(3/2), ln(3/2), ln(3), ln(3), ln(3), ln(3)] = [0, 0, 0, 0.22, 0.22, 0.22, 0.22, 0]

Message 3: [0, 0.25, 0.25, 0.25, 0, 0, 0, 0, 0.25] * [ln(3), ln(1), ln(3/2), ln(3/2), ln(3), ln(3), ln(3), ln(3)] = [0, 0, 0.1, 1, 0, 0, 0, 0, 0.27]

- Message 2 tf: [0, 0.2, 0, 0, 0.2, 0.2, 0.2, 0.2, 0]
- Message 2 idf: [ln(3), ln(1), ln(3/2), ln(3/2), ln(3), ln(3), ln(3), ln(3)]
- Message 2 tf-idf = [0, 0, 0, 0, 0.22, 0.22, 0.22, 0.22, 0]
- Yes, "are good" is equal to because it appears in every sentence, so its idf value is ln(1) = 0. This means that this does not tell us much valuable information in classifying a sentence.
- "are", "good"- these appear in all sentences

- Support is the number of occurrences of each class from the true labels.

- Accuracy falls off as a good metric since the training process will heavily favor the skewed class. A high accuracy won't necessarily mean a high f-1 score since the score also takes into account recall and precision, which may be low as a result of the former. Accuracy is just a measure of all true classifications, while F-1 score penalizes misclassifications. Based on my results, the tf-idf model performed the best with a F-1 score of 0.95 and an accuracy of .9512. Since the supports were relatively balanced, the other models also performed close, and the accuracy and f-1 scores tended to be similar.

Model Improvements (Optional; Not Included in Homework Score)

As we have pointed out in the first class, machine learning pipeline does not end with a single iteration of training, testing and evaluation. Here, we will repeat these process with different hyper-parameters and try to improve the spam detection model so that it performs even better.

Below are two **potential improvements** that may be beneficial to the performance. You are encouraged to complete either (or both) of them. Your response to these questions are optional and won't be graded, but we will provide feedbacks to your code and analysis if you choose to answer them. You can add code and text cells below the questions as needed.

- We use character-level trigrams as tokens during the feature extraction stage, however $N = 3$ is a rather arbitrary choice. What will the performance be like if we use shorter N-grams, such as $N = 1$ and $N = 2$? What about longer N-grams, such as $N = 4$? What if we combine different N-grams, like $N = 1, 2, 3$? Modify the feature extraction code above and play with different (range of) N values. What are the **best (range of) N value on the test set of combined features and SVM classifier**? What metrics did you use to reach this conclusion and why do you think this (range of) N value is the best?

- During the above training and evaluation process, we are only evaluating three combination of feature extractors and classifiers. Can you try to rewrite the training and evaluation code, such that **every combination** of feature extractors and classifiers can be evaluated? What metrics did you use to reach this conclusion and what is the best combination of feature extractor and classifier in terms of test set performance?

References

- CRESCEI-2017 Dataset: <https://botometer.osome.iu.edu/bot-repository/datasets.html>
- TF-IDF: <https://en.wikipedia.org/wiki/TF%E2%80%93IDF>
- N-gram: <https://en.wikipedia.org/wiki/N-gram>
- scikit-learn documentation: <https://scikit-learn.org/stable/modules/classes.html>
- Ridge regression: https://en.wikipedia.org/wiki/Tikhonov_regularization
- SVM: https://en.wikipedia.org/wiki/Support-vector_machine
- Lasso: https://en.wikipedia.org/wiki/Lasso_statistics
- ElasticNet: https://en.wikipedia.org/wiki/Elastic_net_regularization
- Kernel method: https://en.wikipedia.org/wiki/Kernel_method